

Jaypee Institute of Information Technology, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND
INFORMATION TECHNOLOGY



Project Title: NEXUS

Smart Load Balancer

Enrol. No. Name of Student

9923103044 Pratham Arora

9923103119 Divyansh Sharma

Course Name: Fundamentals Of Distributed Cloud Computing

Course Code: 21B12CS313

Program: B. Tech. CSE

3rd Year 6th Sem

2026

Submitted to: Dr. Rashmi Kushwah

INTRODUCTION

The rapid growth of cloud computing and large-scale applications has led to an increasing demand for efficient traffic management and resource utilization. In distributed environments, multiple servers work together to handle incoming requests, making it essential to distribute load effectively to maintain performance and reliability.

Load balancing plays a critical role in ensuring that no single server becomes a bottleneck while others remain underutilized. It helps in improving system throughput, reducing response time, and maintaining high availability. However, understanding how load balancing works in real-world systems is challenging due to the lack of transparency and limited visualization tools.

This project, NEXUS – Smart Load Balancer, is developed to provide a practical and interactive platform for understanding how load balancing operates in distributed systems. It allows users to visualize traffic distribution, experiment with different algorithms, and analyse system performance under varying conditions.

ABSTRACT

NEXUS is a real-time Smart Load Balancer Simulator developed as a project to demonstrate efficient traffic distribution across multiple servers in a dynamic environment. The system models a virtual infrastructure where incoming client requests are distributed among server nodes using various load balancing algorithms such as Round Robin, Least Connections, Weighted Round Robin, IP Hashing, and Least Response Time.

The project is built using a high-performance asynchronous backend that simulates concurrent request handling and real-time system behaviour. Each server node is designed to mimic realistic conditions by incorporating parameters such as processing load, response time, and failure probability. A tick-based simulation engine drives the system, generating traffic, assigning requests, updating server states, and calculating performance metrics at regular intervals.

To enhance system reliability and demonstrate distributed coordination, a simplified consensus mechanism is incorporated to handle control decisions such as leader selection and failure handling. The frontend provides a visually interactive dashboard that displays real-time metrics including Requests Per Second (RPS), latency, and success rates, allowing users to analyse system performance under different scenarios.

The project is containerized and deployed using modern tools, ensuring scalability and ease of execution across environments. By simulating real-world load balancing scenarios and system behaviour, NEXUS provides a practical implementation that reflects key principles of distributed systems such as efficient resource utilization, concurrency, scalability, and fault handling.

PROBLEM STATEMENT

In modern distributed cloud computing environments, efficient load distribution is essential to ensure high performance, scalability, and system reliability. Load balancers are responsible for managing incoming traffic and distributing it across multiple servers; however, their internal working is often complex and not easily observable. Most real-world load balancing systems operate as black boxes, providing only output metrics without offering visibility into how decisions are made internally.

This lack of transparency makes it difficult for students and developers to understand how different load balancing algorithms behave under varying conditions such as traffic spikes, server overload, or node failures. Additionally, there is limited access to platforms that allow real-time experimentation and visualization of distributed system behaviour in a controlled environment.

As a result, there is a need for a system that can simulate a distributed infrastructure, provide real-time insights into traffic distribution, and allow users to analyse the performance of different load balancing strategies. Such a system should also replicate realistic conditions, including server health variations and failure scenarios, to better understand system resilience and efficiency.

The NEXUS project addresses this problem by creating an interactive and real-time simulation platform that models load balancing in distributed systems, enabling users to observe, experiment, and gain practical understanding of system behaviour.

1. BRIEF DESCRIPTION OF PROJECT

NEXUS is a real-time load balancing simulation platform designed to model how incoming network traffic is distributed across multiple servers in a distributed environment. The system creates a virtual setup of server nodes and dynamically generates client requests, which are then assigned using different load balancing algorithms such as Round Robin, Least Connections, and Weighted Round Robin.

The project operates on a time-based simulation engine that continuously processes requests, updates server states, and calculates performance metrics. Each server behaves like a real system by simulating processing delays, varying loads, and failure conditions. This allows users to observe how the system responds to different traffic patterns and stress scenarios.

The platform also includes an interactive dashboard that displays real-time metrics such as request rate, latency, and success rate, helping users analyse system performance. Additionally, basic coordination logic is incorporated to demonstrate how distributed systems handle control and maintain operation during failures.

Overall, NEXUS provides a practical and interactive way to understand how load balancing works in distributed systems by allowing users to visualize and experiment with different strategies in real time.

2. OBJECTIVES OF THE PROJECT

The primary objective of the NEXUS project is to design and develop an interactive platform that demonstrates how load balancing operates within a distributed environment. The project focuses on building a system that allows users to observe, analyse, and experiment with different traffic distribution strategies in real time.

Another key objective is to implement and compare multiple load balancing algorithms, enabling the study of their behaviour under varying traffic conditions and server states. This helps in understanding how different strategies impact system performance, resource utilization, and response time.

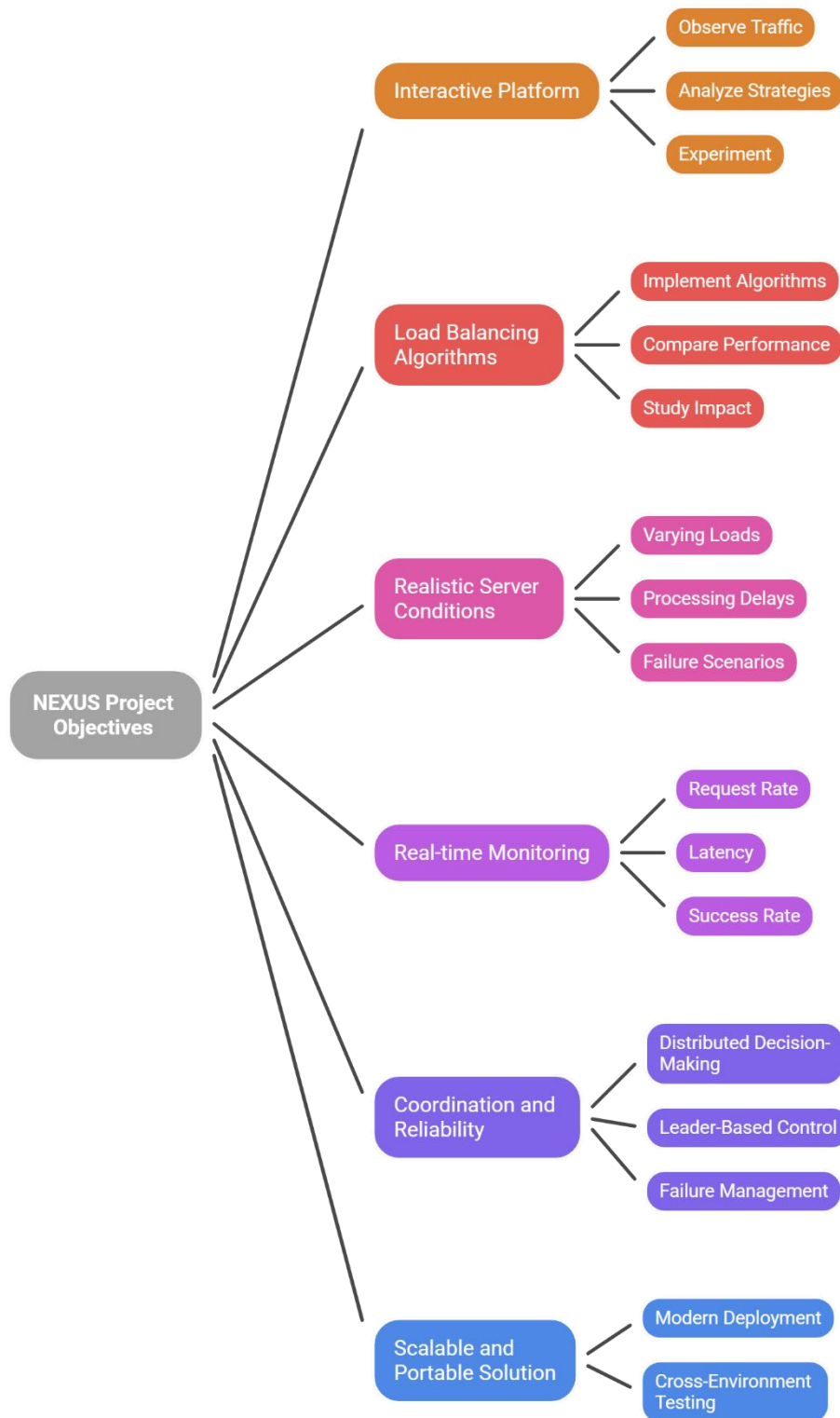
The project also aims to simulate realistic server conditions, including varying loads, processing delays, and failure scenarios, in order to study how systems maintain performance and stability under stress. This provides practical insight into handling dynamic and unpredictable workloads.

An additional objective is to incorporate real-time monitoring and visualization of system metrics such as request rate, latency, and success rate. This allows users to evaluate system efficiency and make informed decisions based on observed data.

Furthermore, the project seeks to demonstrate coordination and reliability mechanisms by introducing basic distributed decision-making concepts, such as leader-based control handling, to reflect how systems manage failures and maintain continuity.

Finally, the project aims to provide a scalable and portable solution using modern deployment practices, ensuring that the system can be easily executed and tested across different environments.

NEXUS Project Objectives



3. IMPLEMENTATION DETAILS

3.1 Tools and Platforms Used

The NEXUS project is implemented using a combination of modern backend technologies, frontend tools, and cloud deployment platforms to simulate a real-time distributed system environment.

Backend Technologies

- **Python 3.10+**

The core simulation engine is developed in Python due to its flexibility and strong support for asynchronous programming. It is used to model server behaviour, request handling, and algorithm execution.

- **FastAPI**

FastAPI is used to build high-performance REST APIs and manage WebSocket communication. It enables asynchronous request handling, which is essential for simulating concurrent traffic in distributed systems.

- **Asyncio**

Asyncio powers the simulation loop, allowing multiple requests to be processed concurrently without blocking execution. This mimics real-world distributed environments where multiple events occur simultaneously.

- **Pydantic**

Used for data validation and serialization of API responses, ensuring structured and error-free communication between backend and frontend.

Frontend Technologies

- **HTML, CSS, JavaScript (Vanilla)**

The frontend is built without heavy frameworks to ensure maximum performance and minimal latency.

- **Chart.js**

Used for real-time visualization of system metrics such as:

- Requests Per Second (RPS)
- Latency
- Success Rate

- WebSockets
Provides real-time, low-latency communication between backend and frontend. The simulation state is continuously pushed to the UI without requiring page refresh.

DevOps and Deployment Tools

- Docker
Docker is used to containerize the application, ensuring consistent execution across different environments. It packages the application along with all dependencies into a single portable container.
- Docker Compose
Used for managing multi-service architecture (e.g., backend + reverse proxy if needed). It simplifies setup and deployment.
- Render (Cloud Platform)
The project is deployed on Render, which provides:
 - Cloud hosting for backend services
 - Automatic deployment from GitHub
 - Scalable infrastructure
 - Environment variable management

3.2 System Implementation Overview

The implementation of NEXUS is based on a modular and decoupled architecture consisting of the following major components:

3.2.1. Simulation Engine

- Central component controlling the system
- Runs on a tick-based loop (every ~500ms)
- Handles:
 - Request generation
 - Load balancing decisions
 - Server updates
 - Metrics calculation

3.2.2. Server Node Model

- Each server is simulated as an independent unit
- Maintains:
 - Active connections
 - CPU load
 - Response time
 - Failure probability
- Dynamically changes state based on incoming traffic

3.2.3. Load Balancing Algorithms

Implemented using a modular strategy pattern:

- Round Robin
- Weighted Round Robin
- Least Connections
- IP Hash
- Random
- Least Response Time

Each algorithm determines how incoming requests are distributed across servers.

3.2.4. API Layer

- Provides control endpoints such as:
 - Start/Stop simulation
 - Traffic spike generation
 - Server addition/removal
- Enables external interaction with the system

3.2.5. WebSocket Communication

- Sends real-time system state to frontend
- Data includes:
 - Server status
 - Metrics
 - Alerts
- Ensures smooth and continuous visualization

3.3 Deployment Using Docker and Render

Docker Implementation

- A Dockerfile is used to define the application environment
- Includes:
 - Python base image
 - Installation of dependencies
 - Copying project files
 - Running the FastAPI server

Benefits:

- Environment consistency
- Easy scalability
- Simplified deployment

Render Deployment

The application is deployed on Render as a cloud-hosted service.

Deployment Process:

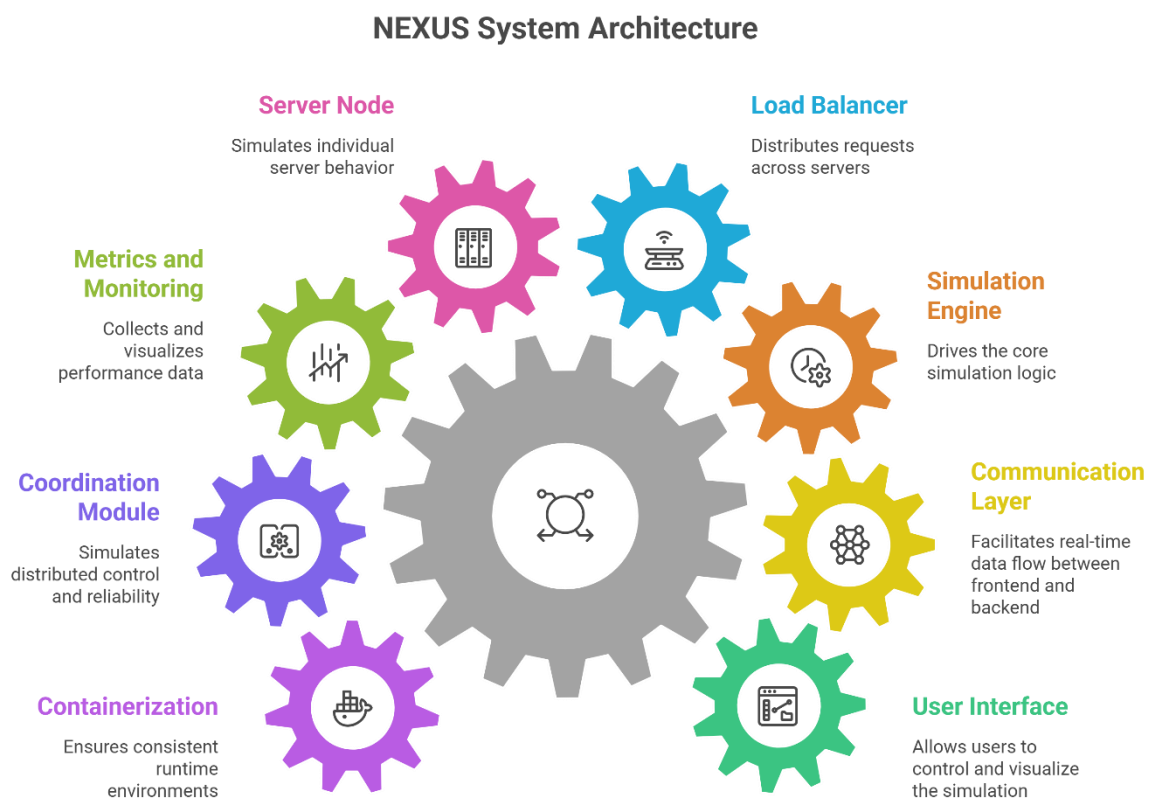
1. Code is pushed to GitHub repository
2. Render connects to the repository
3. Docker container is built automatically
4. Application is deployed as a web service
5. Public URL is generated for access

Features Used:

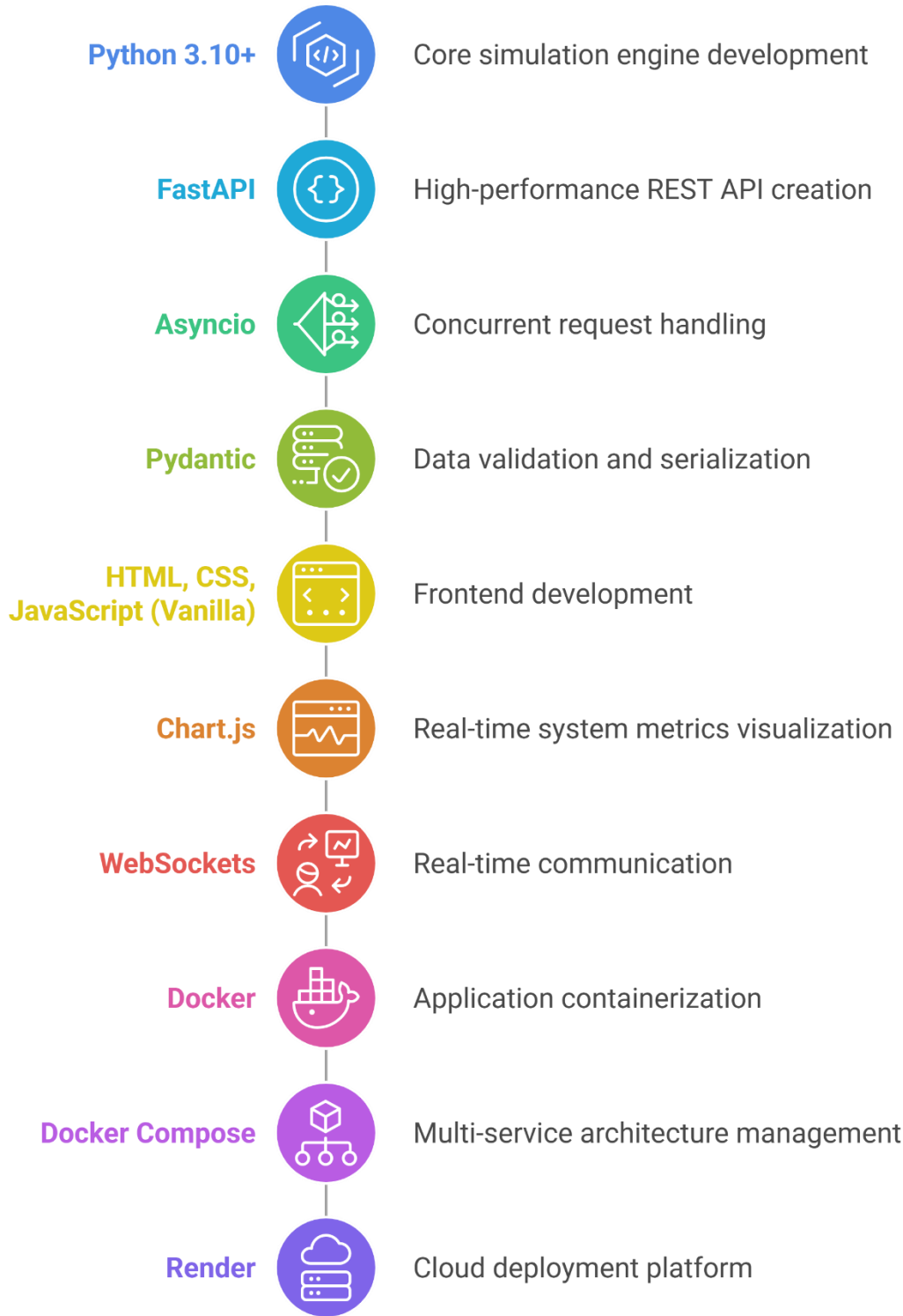
- Auto-deploy on code updates
- Environment variable configuration
- Managed hosting (no manual server setup)

3.4 Why Docker and Render Were Used

- Docker ensures that the application runs identically across all environments, eliminating dependency issues and simplifying deployment.
- Render provides a cloud-based platform that supports containerized applications, allowing easy hosting without managing infrastructure.
- Together, they demonstrate real-world cloud deployment practices, aligning with distributed cloud computing concepts such as:
 - Portability
 - Scalability
 - Reliability



4. NEXUS PROJECT IMPLEMENTATION TIMELINE



5. WORKING METHODOLOGY



User Starts Simulation

The process begins when a user initiates the simulation.

User actions generate requests within the active simulation.

Requests Generated



Algorithm Selects Server

An algorithm evaluates server load and performance to select the best server.

The chosen server processes the incoming request with necessary computations.

Server Processes Request



Metrics Updated

Relevant metrics are updated during and after request processing.

Results are sent back to the client through a WebSocket connection.

Data Sent via WebSocket

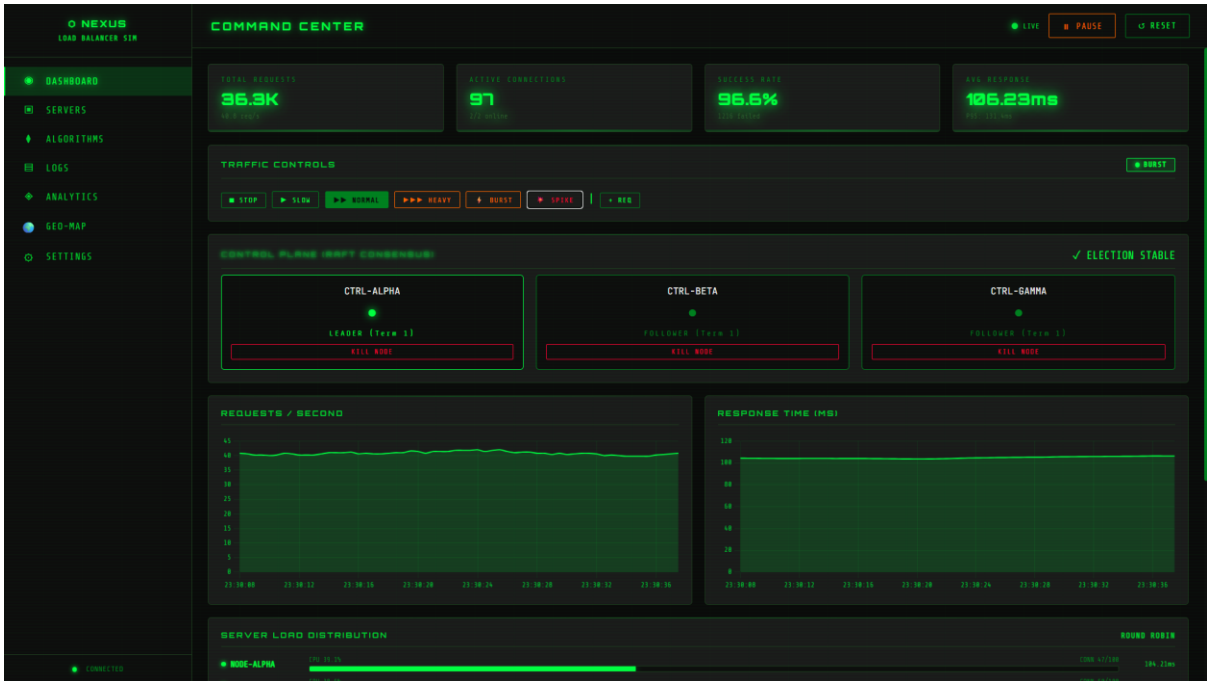


UI Updated

The user interface is updated with the new data from the server.

6. IN APP SCREENSHOTS

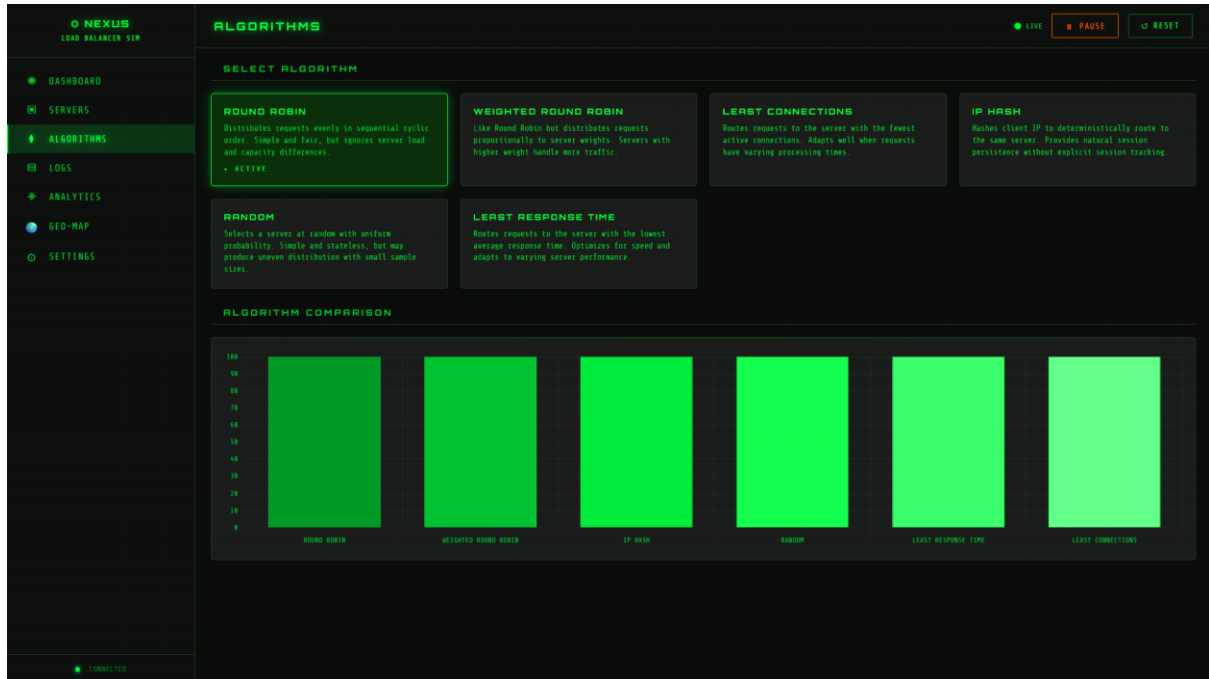
DASHBOARD:



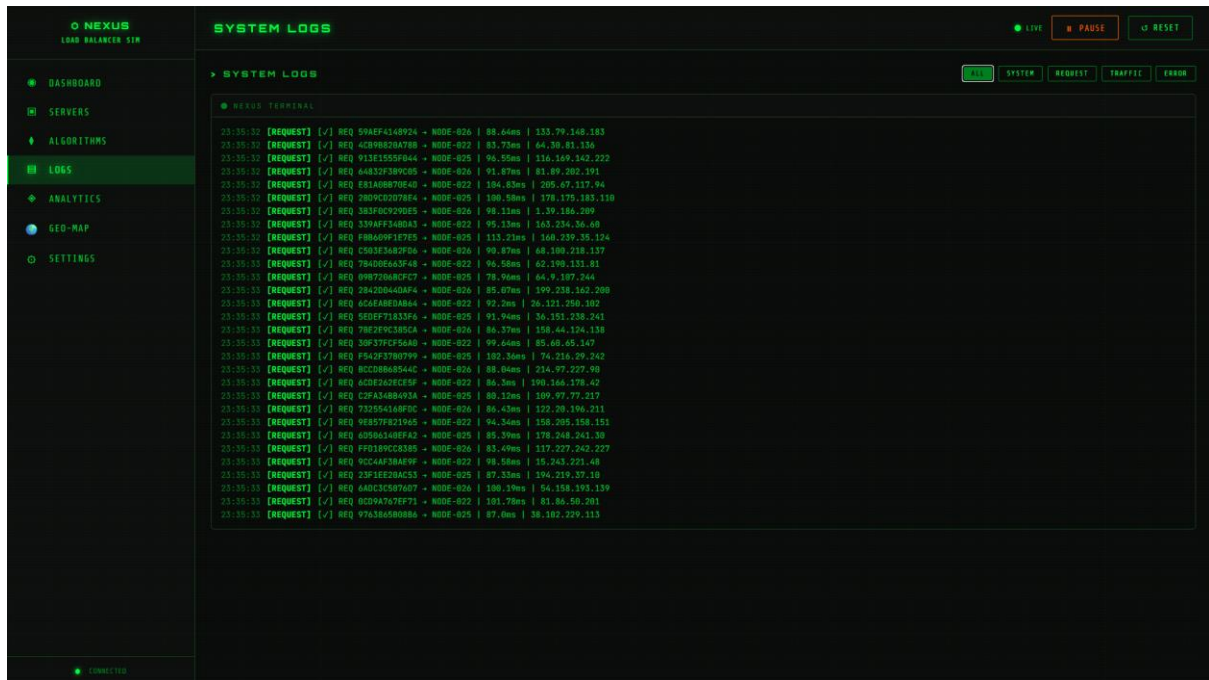
SERVER SCREEN:



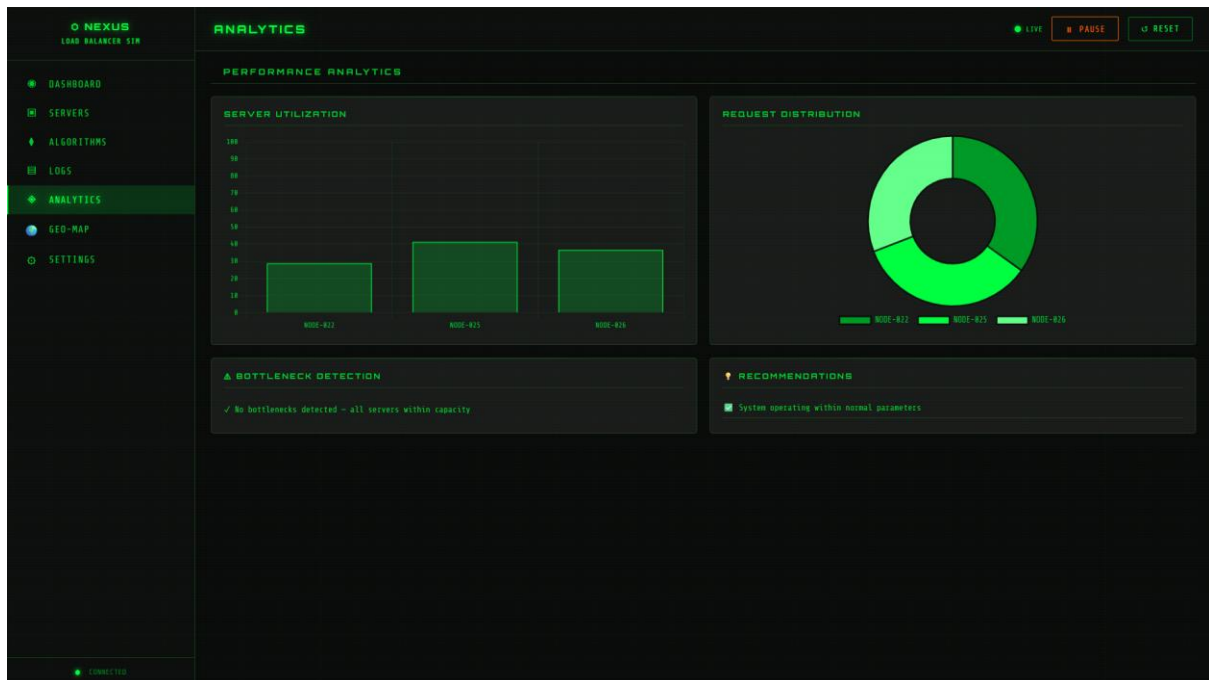
ALGORITHM SELECTION PAGE:



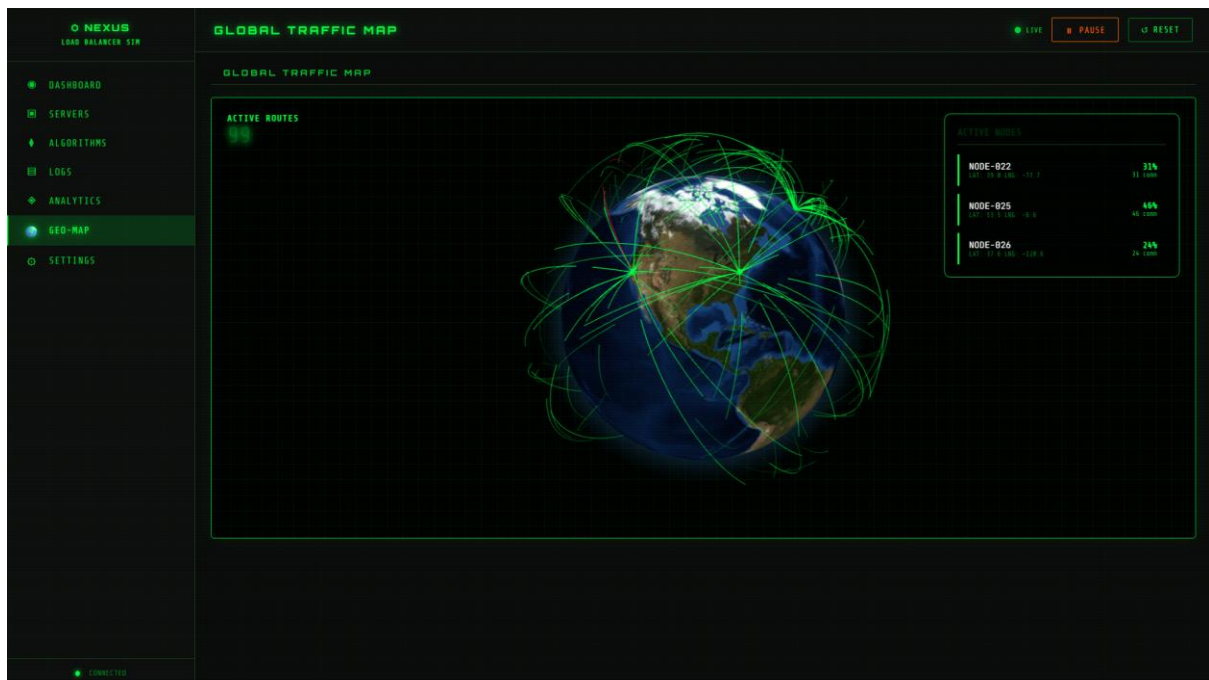
LOGS:



ANALYTICS PAGE:



NODE VISUALIZER (GEO MAP)



SETTINGS PAGE:

NEXUS

LOAD BALANCER SIM

DASHBOARD

SERVICES

ALGORITHMS

LOGS

ANALYTICS

Geo-Map

SETTINGS

CONFIGURATION

▶ CONFIGURATION

SIMULATION SETTINGS

TICK INTERVAL (MS)

500

☒ Sticky Sessions

☒ Auto-Scaling

APPLY SETTINGS

DIRECT FEEDBACK

Have suggestions or found a bug? Send it directly to the architect.

FORM NAME

Agent Smith

EMAIL ADDRESS

smith@nexus.dev

MESSAGE

I've noticed a glitch in the simulation.

SEND TO ARCHITECT

AUTO-SCALE CONFIG

MIN SERVERS

2

MAX SERVERS

8

SCALE UP CPU THRESHOLD (%)

70%

SCALE DOWN CPU THRESHOLD (%)

20%

APPLY SCALE CONFIG

EXPORT DATA

ABOUT NEXUS

NEXUS is a high-performance distributed systems simulator designed by Pratham Azora (DownshifterX).

Built for visualizing load balancing strategies, Raft consensus, and dynamic traffic routing in real-time.

BUILD VERSION: 1.0.0-GOLD

● CONNECTED

7. CONCLUSION

The NEXUS Smart Load Balancer Simulation Platform successfully demonstrates how incoming traffic can be efficiently distributed across multiple servers in a dynamic environment. Through the implementation of various load balancing algorithms, the project provides a clear understanding of how different strategies affect system performance, resource utilization, and response time.

The system effectively simulates real-world conditions such as varying traffic loads, server congestion, and failure scenarios, allowing users to observe system behaviour under different situations. The use of a tick-based simulation engine and asynchronous processing enables realistic modelling of concurrent request handling, which is a fundamental aspect of distributed systems.

Additionally, the integration of real-time visualization through an interactive dashboard enhances the ability to monitor and analyse system metrics such as Requests Per Second (RPS), latency, and success rate. The inclusion of coordination logic further highlights how systems maintain operation and handle failures in a distributed environment.

The deployment of the project using Docker and Render demonstrates practical implementation of modern cloud deployment practices, ensuring portability, scalability, and ease of use. Overall, the project achieves its goal of providing a practical platform to study and analyse load balancing behaviour in a controlled and interactive manner.

8. FUTURE SCOPE

The NEXUS platform can be further enhanced by incorporating advanced features to simulate more complex and realistic distributed cloud environments.

One possible improvement is the implementation of multi-region simulation, where traffic can be routed across geographically distributed data centers. This would allow the study of latency optimization and global load balancing strategies.

Another enhancement could involve integrating machine learning techniques to develop adaptive load balancing algorithms that can predict traffic patterns and automatically optimize resource allocation.

The system can also be extended to support user-defined algorithms, enabling users to create and test custom load balancing strategies for research and experimentation.

Further improvements may include deeper fault tolerance mechanisms, such as advanced consensus protocols and recovery strategies, to better simulate real-world distributed system resilience.

Scalability can be enhanced by increasing the number of simulated nodes and introducing container orchestration tools, enabling large-scale simulation environments.

Finally, the user interface can be upgraded with more detailed analytics, historical data tracking, and performance comparison tools, providing deeper insights into system behaviour over time.

9. REFERENCES

- [1] T. Tiangolo, “FastAPI,” ver. 0.115.0, 2024. [Online]. Available: <https://fastapi.tiangolo.com/> [Accessed: May 2, 2026].
- [2] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Dec. 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455.html> [Accessed: May 2, 2026].
- [3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2011.
- [4] V. Agafonkin, “Leaflet,” ver. 1.9.4, 2023. [Online]. Available: <https://leafletjs.com/> [Accessed: May 2, 2026].
- [5] Docker Inc., “Docker,” ver. 27.0, 2024. [Online]. Available: <https://www.docker.com/> [Accessed: May 2, 2026].
- [6] Render, “Render: Cloud Hosting for Developers,” 2024. [Online]. Available: <https://render.com/> [Accessed: May 2, 2026].
- [7] Resend, “Resend: Email API for Developers,” 2024. [Online]. Available: <https://resend.com/> [Accessed: May 2, 2026].
- [8] Python Software Foundation, “Python Language Reference,” ver. 3.12, 2024. [Online]. Available: <https://www.python.org/> [Accessed: May 2, 2026].
- [9] S. Chacon and B. Straub, *Pro Git*, 2nd ed. New York, NY, USA: Apress, 2014. [Online]. Available: <https://git-scm.com/book/en/v2>.
- [10] Pydantic Team, “Pydantic,” ver. 2.5, 2024. [Online]. Available: <https://docs.pydantic.dev/> [Accessed: May 2, 2026].